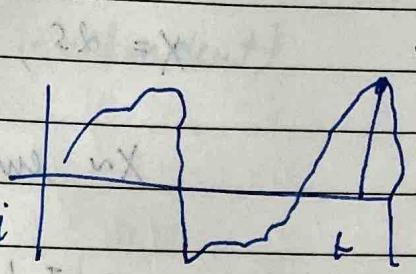
Lecture - 16 - Independent Component Analysis (RL

Independent component Analysis
- CDFs (cumulative distribution function)
- ICA model

Reinforcement learning
- MDP

Sources: $s \in \mathbb{R}^n$



$s_j^{(i)}$ = speaker j at time i

$x^{(i)} = A s^{(i)}$, $x \in \mathbb{R}^n$

Goal: Find $W = A^{-1}$, so $s^{(i)} = W x^{(i)}$ / $W = \begin{bmatrix} — w_1^T — \\ — w_n^T — \end{bmatrix}$

CDF → Cumulative Distribution function

$F(s) = P(S \leq s)$ $\quad P_s(s) = F'(s)$

(↑ Random variable ; ↘ constant)

Later: Choose $F(s)$

Density of s: $P_s(s)$

$x = As = W^{-1}s$

$s = Wx$

$P(x) = P_x(x) = ?$

$p_X(x) = p_S(Wx)$ (This is incorrect).

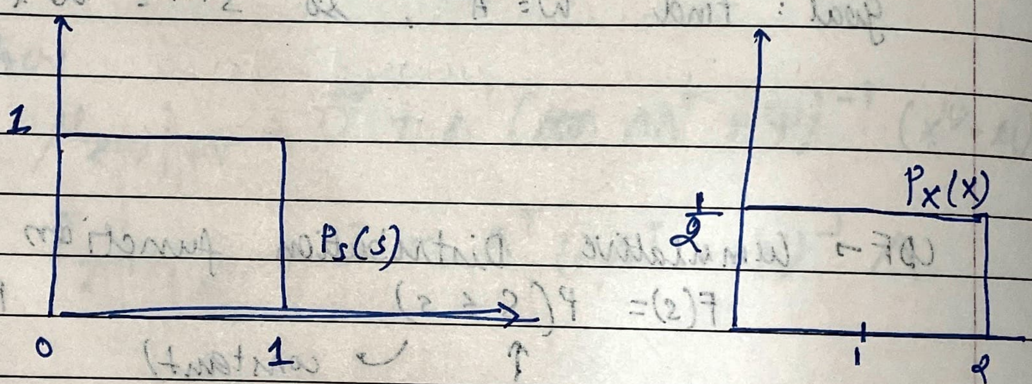
↓ Density of X (under Wx: p_S)

$p_S(s) = I\{0 \le s \le 1\}$ (i.e. $S \sim$ uniform$(0,1)$)

$X = 2S$, ($A = 2$, $w = 1/2$)

$X \sim$ uniform $(0, 2)$

$$p_X(x) = \frac{1}{2} I\{0 \le x \le 2\}$$



so correct formula would be

$$p_X(x) = p_S(Wx) \cdot |W|$$

↑ Density of X; p_S; $|W|$: determinant of matrix W

$P_s(s) = ?$

$F(s) = P(S \leq s) = \frac{1}{1+e^{-s}}$

$P_s(s) = F'(s)$

$$P(s) = \prod_{i=1}^{n} P_s(s_i)$$

(n speakers are independent)

$$P_x(x) = P_s(w_x) \, |w|$$

$$= \left| \prod_{j=1}^{n} P_s(w_j^T x) \right| |w|$$

MLE → Maximum Likelihood Estimation - to estimate parameters w

$$l(w) = \sum_{i=1}^{m} \log \left(\prod_{j} P_s(w_j^T x^{(i)}) \right) |w|$$

Stochastic gradient ascent →

$$\nabla_w l(w) = \begin{bmatrix} 1 - \alpha g(w_1^T x) \\ \vdots \\ 1 - 2g(w_n^T x) \end{bmatrix} x^{(i)T} + (w^T)^{-1}$$

Reinforcement Learning

$R(s)$ = reward of state
- $+1$ for win
- -1 for lose
- 0 for tie

Markov Decision Process (MDP) $\Rightarrow (S, A, \{P_{sa}\}, \gamma, R)$

S - set of states

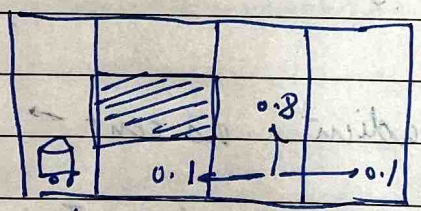
A - set of actions

P_{sa} - state transition probabilities

$$\sum_{s'} P_{sa}(s') = 1$$

γ - discount factor $\gamma \in [0, 1)$

R - reward function



S - 11 states

$A : \{N, S, E, W\}$

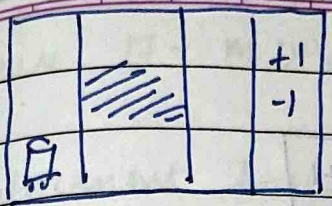
$P_{(3,1)N}((3,2)) = 0.8$

$P_{(3,1)N}((4,1)) = 0.1$

$P_{(3,1)N}((2,1)) = 0.1$

$P_{(3,1)N}((3,3)) = 0$

Suppose the prize is at $(4, 3)$ so we assign it a reward of $+1$ & we definitely dont want the robot to go at $(4, 2)$, so we assign it a reward of -1



$R((4,3)) = +1$
$R((4,2)) = -1$
$R((s)) = -0.02$ for all other states

The robot starts at s_0
Choose action a_0 (N, S, E, W)
Get to $s_1 \sim P_{s_0 a_0}$

Choose action a_1
Get to $s_2 \sim P_{s_1 a_1}$

Total pay off =>
$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) \ldots$$

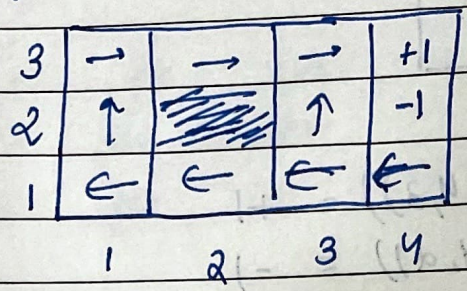
$\gamma = 0.99$ (chosen slightly less than 1).

Goal: Choose actions over time to maximize total pay off.

$$E[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) \ldots]$$

Policy $\pi : S \longmapsto A$
(controller)
Optimal policy: When in state s, take action $\pi(s)$

Optimal policy:



$$\Pi((3,1)) = W$$

This is how we formulate a problem as an MDP, then its the job of the reinforcement learning algorithm is to go through RDP & tell you which is a good policy.

In above if we start at ((3,1)), we have 2 possible paths to traverse to reach the reward.